BUKIDNON STATE UNIVERSITY
College of Technologies
Information Technology Department

Academic Early Warning System

Test Plan and Schedule

IT143 CO3 Major Assessment
Quality Assurance Project Management Framework

Document control	Details
Identifier / version	AEWS-QA-TP-001 / 2.0
Preparation date	7 November 2026
Requirements reference	AEWS-QA-RR-001, Requirements Review Report, version 2.0
Associated Trello task	Card 2 - Prepare AEWS Test Plan and Schedule
Status	Proposed plan; team assignments, calendar and approval pending

Plan overview

This plan defines the scope, methods, responsibilities, resources, evidence and schedule for quality assurance of the Academic Early Warning System (AEWS). It aligns seventeen requirement groups with forty prepared test cases and adds targeted scenarios for the seven findings in the requirements review.

Execution follows a proposed seventeen-working-day schedule with explicit dependencies, initial testing, defect resolution, regression and final review. Actual calendar dates and member assignments are recorded in editable fields. No testing, defect resolution or approval is represented as completed by this planning document.

The plan supports the assessment guidelines and rubric [1-2]. Its proposed quality targets and internal schedule are not instructor-mandated deadlines.

1. Objectives and Test Scope

1.1 Objectives

Verify that AEWS preserves academic-data accuracy, enforces identity and record permissions, supports a traceable early-intervention workflow and produces consistent reports. Detect defects before the Capstone evaluation, provide reproducible evidence, and support an explicit readiness decision based on observed results.

1.2 In-scope requirements

Requirement coverage	Verification focus	Main cases
R01-R03: identity and access	Registration, approval, session/recovery, role and ownership boundaries	TC-01 to TC-08
R04-R05: classes and imports	Class lifecycle; supported files; identity matching; invalid and duplicate input	TC-09 to TC-15
R06-R07: academic accuracy	Term/component separation, grade validity, attendance calculation and missing data	TC-16 to TC-18
R08-R09: referrals/assessment	Referral boundaries, student linkage, form validation and invitation lifecycle	TC-19 to TC-24
R10-R11: prediction/routing	Input sufficiency, provenance, output meaning and controlled communications	TC-25 to TC-30
R12-R13: student/admin	Own-student profile/dashboard; account/form changes and audit behavior	TC-31 to TC-34
R14: reports	Role-scoped totals, filters, exported content and layout	TC-35 to TC-36
R15-R16: quality attributes	Task usability, keyboard/narrow viewports and measured response times	TC-37 to TC-39
R17: integration	Complete intervention workflow and fix-related regression	TC-40 and related cases

1.3 Exclusions and boundaries

Exclude unapproved production changes, university-wide load certification, unrestricted penetration testing and claims of real-world student-outcome effectiveness. The planned security checks use controlled test accounts and isolated data. Model accuracy is assessed only through documented evaluation evidence; successful inference does not establish predictive accuracy.

The approved specification governs expected results. The requirements review [3] identifies source observations and proposed requirements; unresolved policy decisions must be clarified before affected tests receive a pass/fail outcome.

2. Test Strategy and Execution Method

2.1 Risk-led sequence

Begin with baseline review and environment readiness, followed by account/ownership isolation and import accuracy. Establish trustworthy academic fixtures before validating referrals, assessments, prediction and intervention. Exercise reporting and quality attributes on a stable build. Reserve explicit time for fixes, retesting and integration regression.

Method	Application and evidence
Static review	Resolve requirement ambiguity, review access/data contracts and confirm expected results before execution. Link decisions to R01-R17 and findings F01-F07.
Functional and integration	Test permitted role workflows through the interface and corresponding API. Record expected/actual behavior and persisted state where relevant.
Boundary and negative	Use valid/invalid credentials, referral limits, zero/missing values, invalid file types/sizes, duplicates, expired invitations and out-of-scope record IDs.
Independent reconciliation	Compare uploaded identities, grades, attendance, totals and export rows with independently prepared expected values.
Automated checks	Run existing backend/frontend suites and production build checks in the configured test environment. Retain logs and build identity; automation complements manual business-rule tests.
Retest and regression	Repeat original failed steps on the fixed build, verify boundary variants and rerun affected downstream functions before closing a defect.

2.2 Execution record

For each scenario, record the requirement/case ID, preconditions, exact input, build, environment, tester, date, expected result, actual result, status, sanitized evidence and linked defect. The case workbook supplies forty grouped procedures [4]. Give independent variants unique execution suffixes, such as TC-19-A and TC-19-B, rather than combining conflicting outcomes into one result.

2.3 Status definitions

Not run: execution has not occurred. Pass: observed outcome meets every applicable expected condition. Fail: an observed outcome differs from the approved expectation. Blocked: a prerequisite, environment condition or unresolved requirement prevents valid execution. A skipped or blocked case is never counted as a pass.

3. Targeted Security, Data and Model Checks

The requirements review identifies seven open findings [3]. The following supplemental scenarios extend the existing cases; they are planned checks, not executed findings or confirmed deployment behavior.

Review finding	Required scenario / expected verification
F01: trusted identity	Extend TC-06/07 with missing, invalid and expired tokens and untrusted user/role headers. Client-declared identifiers must not grant protected authority.
F02: student ownership	Extend TC-08/31/32 with no-session requests and another synthetic student identifier. Private reads and updates require approved identity and scope.
F03: prerequisites	Extend TC-25/26/28 for each missing source at each prediction entry point. Enforce the approved workflow-specific matrix; preserve valid zero and false/no values.
F04: grade domain	Extend TC-17/19/25 with zero, missing, negative, non-finite and above-range grades through import/update/inference paths. Apply the approved grade-domain policy.
F05: output semantics	Extend TC-27/35 to distinguish predicted risk, intervention-cause labels, probability scale and model/fallback source across API, interface and reports.
F06: quality targets	Approve workload, usability tasks and model/performance criteria before TC-27/37/38/39 is assessed for acceptance.
F07: lifecycle policies	Extend TC-10/13/24/34 for partial imports, duplicates, invitation expiry/reuse and existing invitations after form edits.

3.1 AEWS-specific data distinctions

Test the observed whole-midterm automatic referral interval of 2.50 through 5.00 inclusively only after confirming it against the approved academic policy. Test below the threshold and invalid above-range input. Do not equate a referral event with a predicted High Risk result. Attendance 0% is a valid observation; a missing attendance source is different.

3.2 Model evaluation protocol

Record artifact/profile/version, feature order, dataset origin and evaluation run. Examine train/test separation, student/duplicate leakage and outcome leakage. Report held-out sample counts, class distribution, confusion matrix, High Risk recall, precision and F1 with accuracy. Review fallback behavior separately. Record limitations of synthetic data and do not present historical stored metrics as newly reproduced results [6].

4. Test Environment, Data and Resources

4.1 Environment baseline

Use a separate test database, controlled accounts and a fixed source/build version. Record React frontend and FastAPI service versions, browser/OS, database configuration and model artifact. Secrets are stored privately and are not entered in public attachments. Reset state between destructive/lifecycle variants to avoid test-order dependence [5-6].

Source/build, frontend/API test locations and database alias

Browser/OS, device/viewport and performance resources

Model/profile/version and intercepted or controlled email setup

4.2 Synthetic accounts and fixtures

Prepare two instructors with separate classes, an administrator, appropriate AMU staff and two student identities for cross-owner tests. Use genuine test-role assignments, not credentials copied from production. Intercept or disable external email for file-upload/referral scenarios.

Card 5 fixture	Purpose / expected values
Synthetic_Classlist.xlsx	Five invented student IDs and names using example.com addresses; load first into an isolated class.
Synthetic_Gradesheet.xlsx	Matching IDs; MTG values 2.25, 2.49, 2.50, 3.00 and 5.00. Load after roster preparation.
Synthetic_Attendance.xlsx	Five marked sessions; expected attendance 100%, 80%, 60%, 0% and 100%.
Negative fixtures	Duplicate classlist and unsupported text file. Add malformed, empty, mismatched and size-boundary fixtures for relevant cases.

4.3 Preparation and cleanup

Confirm fixture parsing in the current build; reconcile rows before using them as downstream inputs. For reports use known-count data; performance tests require a separately prepared 100-student synthetic class. Record reset/cleanup actions and retain only sanitized evidence needed for traceability.

Readiness check / environment owner / date / outstanding dependencies

5. Responsibilities and Coordination

Assign actual names to the functions below before scheduling execution. Roles may be combined in a four- or five-member group. A person may develop and test different modules, but another member should review their own fix evidence where practical.

Responsibility	Accountability
QA coordinator	Maintains plan, dependencies, milestones, triage, reporting and final review coordination.
Accounts/access tester	Covers authentication, role/ownership isolation, student portal and administrative state changes.
Academic-data tester	Covers classes, imports, grade/attendance reconciliation and data fixtures.
AMU/model tester	Covers referrals, assessments, prerequisite rules, prediction provenance and support routing.
Reports/quality tester	Covers dashboard/export reconciliation, usability, response-time measurement and integration regression.
Responsible developer	Investigates assigned defects, supplies fix/build reference and communicates impact; tester verifies closure.

5.1 Actual assignments

QA coordinator / plan reviewer

Accounts/access tester and reviewer

Academic-data tester and reviewer

AMU/model tester and reviewer

Reports/quality tester and reviewer; shared assignment if needed

5.2 Working practice

Review active tasks and blockers each working day. Log decisions and progress on the appropriate Trello card. Triage critical/high issues promptly and revise the schedule when a prerequisite is delayed. The coordinator tracks due dates; every member maintains their assigned evidence and comments.

6. Proposed Work Schedule and Dependencies

The schedule is a planning baseline of seventeen working days, beginning on the team-selected start date. Working days exclude non-working periods selected by the group. They are not calendar dates or official submission deadlines. Parallel work assumes sufficient members and test resources; the coordinator must adjust it to actual capacity.

Card / activity	Working days	Dependency and planned output
1 Requirements review	1-2	Project artifacts; resolve baseline decisions. Output: Reviewed requirements / issue dispositions.
2 Test plan and schedule	1-2	Initial Card 1 scope; joint final review. Output: Approved methods, owners and dates.
3 Test cases and fixtures	2-4	Reviewed Cards 1-2; environment readiness. Output: Cases and synthetic data ready.
4 Accounts and access	5-10	Card 3; student scenarios after referral fixtures. Output: Account/role/student/admin results.
5 Classes and uploads	5-7	Card 3; usable test identity/class setup. Output: Reconciled baseline imports.
6 Grades and attendance	6-8	Card 5 baseline records available. Output: Independent data comparisons.
7 Referrals and assessments	8-9	Cards 5-6; approved trigger/form policy. Output: Referral and assessment results.
8 Prediction and routing	9-10	Card 7 complete source fixtures. Output: Prediction/routing results.
9 Dashboards and quality	9-12	Relevant Cards 5/7/8 data; stable build. Output: Report/usability/performance results.
10 Fixes and regression	5-15	Observed defects; fixes; final flow after 8-9. Output: Retest evidence; TC-40 regression.
11 Final QA review	16	All results accounted for; Card 10 review. Output: Documented readiness decision.
12 Presentation/submission	17	Current Card 11 decision; accurate board. Output: Walkthrough, link and screenshots.

6.1 Dependency and capacity rules

Cards 1 and 2 may be drafted concurrently but must be jointly reviewed before execution. Prepare accounts needed for imports before Card 5. Card 6 may start after a validated subset of Card 5 data is available. Card 7 requires the approved academic baseline; Card 8 requires complete assessment/source fixtures. Card 9 starts available dashboard checks early and completes downstream reporting after Card 8.

Card 10 runs throughout execution for early triage and fixing. Days 13-15 are reserved for final fix verification and affected regression. Do not assign one member overlapping tasks beyond their capacity. A delayed prerequisite moves dependent dates; it must not be bypassed to preserve the original calendar.

7. Calendar Commitments and Milestones

Enter real calendar dates in YYYY-MM-DD format and copy them to each Trello card's Dates field. Use actual member names under Members. These editable fields define the committed calendar once reviewed; an empty field is not an assigned deadline.

Start date / Capstone milestone / assessment submission deadline

Card / activity	Start date	Due date
01 Requirements review		
02 Test plan and schedule		
03 Cases and fixtures		
04 Accounts/access		
05 Classes/uploads		
06 Grades/attendance		
07 Referrals/assessment		
08 Prediction/routing		
09 Dashboards/quality		
10 Fixes/regression		
11 Final review		
12 Presentation/submission		

7.1 Milestone acceptance

Working day 2: requirements and test plan reviewed. Day 4: cases/environment ready. Day 12: initial execution recorded, including blocked/not-run cases. Day 15: fixes/retesting/regression reviewed. Day 16: readiness decision recorded. Day 17: board checked and each member submits the required link and screenshot.

Calendar reviewed by / review date / approved adjustments

8. Entry, Exit, Suspension and Resumption Criteria

8.1 Entry criteria

Begin a test only when its expected behavior is reviewed, required accounts/fixtures exist, the selected build is reachable, evidence capture works and the environment is isolated from production. Resolve requirements findings affecting the expected outcome or explicitly record the case as Blocked. Confirm prerequisites separately for each prediction entry point.

8.2 Test and task completion

A case passes only when all expected conditions are observed with reproducible evidence. A task card may be completed when its agreed execution/reporting output is reviewed, even if product defects remain open on linked cards. This does not approve the system. Card checklists and dates must reflect actual work.

8.3 Proposed overall QA exit criteria

Criterion	Required evidence
Coverage accounted for	All scoped cases and supplemental scenarios classified as Pass, Fail, Blocked or Not run, with reasons for exclusions.
Core workflows verified	Access/ownership, import integrity, academic calculations and core intervention flow pass on the reviewed build.
Release blockers controlled	No unresolved Critical/High defect compromising private data, academic integrity or a core intervention function. If one remains, record Not ready.
Fixes independently checked	Original failures and affected regressions rerun; reopening and limitations recorded.
Quality and residual risks reviewed	Approved targets assessed; model limitations disclosed; lower-severity residual issues accepted by a named authorized reviewer.
Final decision recorded	QA summary, build, reviewers, date and rationale present; no implied or fabricated approval.

8.4 Suspend and resume

Suspend affected tests for database instability, incorrect fixtures, unavailable essential services, unsafe real-data exposure, invalid build identity or a blocker preventing meaningful observation. Preserve logs and mark the affected tests Blocked. Resume after the cause is corrected, the environment is rechecked, scope is reassessed and impacted earlier tests are selected for regression.

Submission readiness and product readiness are separate decisions. The group may submit an accurate board showing unresolved work; it must not relabel failures or invent approvals to meet the submission date.

9. Defects, Metrics and Quality Measurement

9.1 Defect lifecycle

Create a linked BUG record containing case/requirement, build, steps, expected and actual outcomes, impact, evidence, severity, priority, developer and target date. Workflow: Open, In progress, Fixed awaiting verification, Verified or Reopened. An accepted limitation remains a documented residual issue, not a verified fix.

Severity	Impact definition
Critical	Private-data exposure, major data loss/corruption or total core service failure.
High	Incorrect or unavailable core function without an acceptable workaround.
Medium	Partial failure with an available workaround or limited scope.
Low	Minor presentation or wording issue without material functional impact.

9.2 Execution metrics

Planned = Pass + Fail + Blocked + Not run, using one consistent scenario-counting unit. Executed = Pass + Fail. Execution rate = Executed / Planned x 100. Pass rate among executed = Pass / Executed x 100. Use N/A when a denominator is zero. Report failed, blocked and unexecuted coverage openly; do not convert it into passing coverage.

9.3 Performance protocol and proposed targets

Use a 100-student synthetic class, recorded device/network/browser/server conditions and a single-user baseline. Measure twenty runs per selected dashboard, search and single-student prediction task. Declare the timing boundary, distinguish cold from warm runs, and report median and nearest-rank p95 together with errors/timeouts. Sort twenty observed durations; nearest-rank p95 is the nineteenth duration.

Proposed acceptance: p95 dashboard/search at or below 3 seconds and p95 single-student prediction at or below 5 seconds. These limits require team approval before execution. Report cold-start observations separately and retain failures; do not replace failed requests with zero durations. Record upload size/rows and duration separately.

Approved targets / workload changes / approver and date

9.4 Usability and model outcomes

Document defined task completion, confusing steps, keyboard/focus behavior, validation clarity and viewport coverage. Do not invent satisfaction ratings. Model evidence follows Section 3.2; no accuracy threshold is assumed without an approved requirement.

10. Evidence, Deliverables and Trello Monitoring

Card	Planned attachment / evidence
1	Academic Requirements Review Report and actual review dispositions
2	This Test Plan and Schedule with actual assignments/dates
3	AEWS_Test_Cases.xlsx with reviewed procedures and scenario extensions
4	Account/access results including student/admin checks; actual screenshots
5	Class/upload results and synthetic valid/negative fixture files
6	Grade/attendance reconciliation workbook; actual evidence
7	Referral/assessment results and relevant form evidence
8	Prediction/routing results and model evaluation record
9	Dashboard/quality results, export validation record and actual system export
10	Bug/retest register and end-to-end regression record
11	Final QA summary and actual readiness decision
12	Presentation script, actual readable board screenshots and submission checks

10.1 Evidence management

Use filenames identifying the case, build and date, for example TC-19-A_referral_boundary_buildID_date.png. Record the unmodified expectation separately from observed results. Store sensitive originals privately if required; publish only synthetic or sanitized evidence. Keep logs free of passwords, tokens, connection strings and confidential student information [1].

10.2 Board operation

Use Backlog, To Do, In Progress, Testing and Completed. Assign actual Members and Dates. Use category labels, genuine progress comments, linked defects and useful attachments. A reviewer checks the output before completion. Mark dependency delays in comments and revise affected dates.

10.3 Presentation and submission

Explain the structure, owners, deadlines, QA choices and actual status using one representative detailed card. Each member individually submits the shared public board link and readable screenshot as required by the activity. Verify signed-out access only after removing sensitive content [1-2].

11. Risks, Change Control and Plan Approval

Risk	Response and accountable function
Requirements remain ambiguous	QA coordinator obtains decisions; block affected expected outcomes and update traceability.
Incorrect academic mapping	Academic-data tester reconciles source identities/terms before downstream tests.
Private-data or role leakage	Access tester prioritizes isolation checks; suspend unsafe affected workflows and record blocker.
Prediction appears valid with missing data	AMU/model tester verifies sources, output meaning and model/fallback provenance.
Service/email/model unavailable	Environment owner uses controlled services, documents blocked cases and restores prerequisites.
Schedule overruns or fix regressions	Coordinator reserves retest days, triages daily and rebaselines dependencies honestly.

11.1 Change control

Record changes to scope, requirements, cases, targets, owners, environment or dates with a reason, affected IDs, reviewer and decision date. Reassess dependencies and regression needs. Retain previous versions and execution history; do not overwrite failed evidence with a later pass.

11.2 Actual review and approval

Team / prepared by

Plan reviewer(s) and roles

Decision: approve / approve with conditions / revise and resubmit

Conditions, approved changes and unresolved dependencies

Actual acknowledgement / review date / approved plan version

The plan is not approved until the project reviewers record their actual decision. This acknowledgement does not imply an instructor signature or approval.

12. References and Planning Basis

[1] Bukidnon State University, College of Technologies, Information Technology Department. (n.d.). IT143 CO3 major assessment guidelines: Quality Assurance Project Management Framework using Trello. Supplied PDF, pp. 1-3.

[2] Bukidnon State University, College of Technologies. (n.d.). Rubric for group activity using Trello: Quality Assurance Project Management Framework/timeline creation. Supplied PDF, p. 1.

[3] AEWS project group. (2026). Requirements Review Report (AEWS-QA-RR-001, version 2.0). Requirement groups R01-R17 and review findings F01-F07. Static review; stakeholder validation pending.

[4] AEWS QA preparation materials. (2026). AEWS_Test_Cases.xlsx, TC-01 to TC-40, and card-specific result/evidence templates. Prepared procedures; not evidence that tests have been executed.

[5] AEWS project repository. (n.d.). README.md; frontend/package.json; frontend/src/App.jsx; backend/app/authz.py; backend/app/main.py; backend/app/routers/classes.py, students.py, amu_staff.py and admin.py. Local project artifacts used to establish scope and candidate test paths.

[6] AEWS project repository. (n.d.). backend/app/ai_model.py; backend/MODEL_DOCUMENTATION.md; backend/tests/test_amu_prediction_requirements.py. Prediction source, model documentation and existing prerequisite tests. Their presence does not establish passing runtime behavior.

Supporting checks available in the repository

After the development environment and dependencies are configured, run the backend suite with python -m pytest backend/tests from the repository root. In the frontend directory, use npm test and npm run build. Record output, environment and build. Setup errors are distinguished from product defects. These commands are planned supporting checks and were not run to produce this document.

Planning limitations

No actual team calendar, member names, test environment details or signed requirements baseline were supplied. Consequently, actual assignments and dates remain editable, and the seventeen-day schedule and numerical targets require review. The plan makes no claim of live-system test completion, model accuracy, defect closure or deployment readiness.